Divide-and-conquer sorting algorithms

This week we'll consider two fast algorithms for sorting. Along the way, we'll illustrate some important ideas that recur in algorithm design:

- theoretical lower bounds on possible algorithms for problems
- using the divide-and-conquer strategy to get fast algorithms
- the difference between average and worst case performance

By the end of this session, you should be able to:

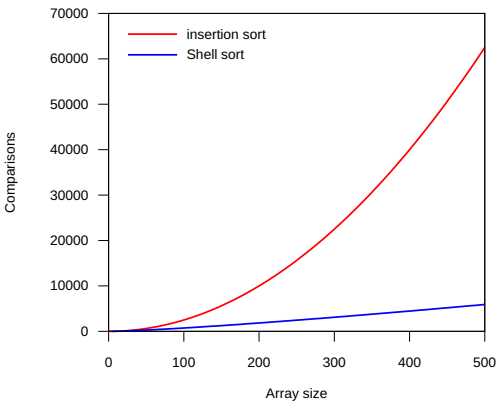
- describe the general idea of finding theoretical lower bounds, and how it is applied to sorting
- specify the advantages and disadvantages of quicksort and merge sort
- show how these algorithms operate on sample data

How fast can a sorting algorithm be?

In week 3, we considered the selection sort and insertion sort algorithms, which are both $O(n^2)$. We more briefly considered Shell sort, which, though difficult to analyse, is much faster. It is natural to wonder how far we can go.

This is a common situation in computer science: there may be several algorithms for a problem, but we would like to establish a *lower bound* on their cost. If we have an algorithm that achieves that complexity, we say it is *asymptotically optimal*.

> The term *asymptotic* refers to what happens when the input size becomes large, which we express using big-O notation.



Experimentally measured average-time performance of insertion sort and Shell sort

Establishing lower bounds requires different approaches. We'll consider sorting below. For some other problems this is more difficult (more on this next week).

Counting comparisons

Let's take the comparison of two keys as the basic operation. In the abstract:

- How many ways can a sequence of n elements be arranged?
- If we could divide that number in half each time we compared two keys, how many would it take to reduce it to the right one? (compare binary search)

We're assuming that we're sorting using comparison of keys, so this analysis doesn't address more esoteric methods (e.g. radix sorting).

Let's try some small examples:

Algorithm Sort1(a)

```
RETURN a
```

1 possible arrangement
0 comparisons

Algorithm Sort2(a, b)

```
IF a < b
    RETURN (a, b)
ELSE
    RETURN (b, a)
```

2 possible arrangements
1 comparison to distinguish between them

Algorithm Sort3(a, b, c)

```
IF a < b
    IF b < c
        RETURN (a, b, c)
    ELSE        c ≤ b
        IF a < c
            RETURN (a, c, b)
        ELSE        c ≤ a
```

6 possible arrangements (1*2*3)
3 comparisons (at most) to distinguish between them

$$1*2*3 = 6 \leq 2^3$$

or equivalently

$$\log(1*2*3) = \log(6) \leq 3$$

```
                RETURN (c, a, b)
ELSE        b ≤ a
        IF b < c
                IF a < c
                        RETURN (b, a, c)
                ELSE        c ≤ a
                        RETURN (b, c, a)
        ELSE        c ≤ b
                RETURN (c, b, a)
```

If there are 4 elements, there are 1*2*3*4 = 24 permutations, and we will need up to 5 comparisons to distinguish them, because

$$1*2*3*4 = 24 \leq 2^5 \quad \Leftrightarrow \quad \log(1*2*3*4) = \log(24) \leq 5$$

In the general case, we are sorting n elements. To work out the number of permutations of these, we note that there are n choices for the first element, n-1 choices for the second, and so on, yielding a total number of n*(n-1)*(n-2)* ... *3*2*1. This is conventionally called the *factorial* of n, written n!.

So there are n! possible arrangements of n elements. If each comparison allows us to halve the number of possibilities, we can narrow down to the right one in no more than log(n!) comparisons (rounded up to the nearest whole number). An algorithm that could achieve that would be optimal.

Approximation

We are interested in big-O complexity, and it turns out that we can simplify O(log(n!)). First, we note that

$$n! = 1*2*3*...*(n-1)*n \leq n*n*n*...*n*n = n^n$$

and so we have

$$\log(n!) \leq \log(n^n) = n \log n$$

This crude approximation is sufficient for big-O notation. For a better one, look up Stirling's approximation.

And thus log(n!) is O(n log n). But is this the best we can do? We also have

$$n! = 1*2*3*...*(n-1)*n \geq (n/2)*...*(n-1)*n \geq (n/2)*...$$
$$(n/2)*(n/2) = (n/2)^{n/2}$$

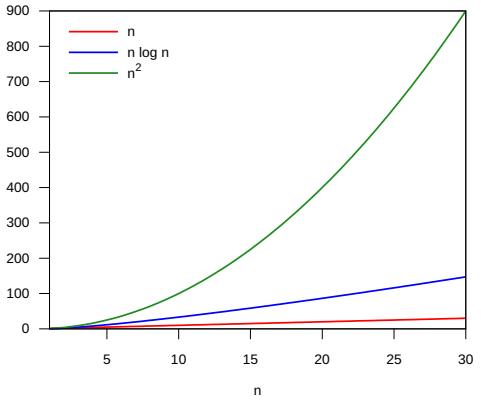
and thus

$$\log(n!) \geq \log((n/2)^{n/2}) = (n/2)\ (\log n - 1)$$

which is also O(n log n), so we have shown

$$O(\log(n!)) = O(n \log n)$$

Thus no sorting algorithm can do better than O(n log n) comparisons in the worst case (or the average case). A sorting algorithm that achieves that is said to be *asymptotically optimal*. Several such algorithms are known; we'll consider two of them.



O(n log n) is only a little worse than O(n), and much better than $O(n^2)$.

For some problems, there is a gap between the best known algorithm and the tightest known lower bound. Perhaps there's a better algorithm to be found, but no-one knows. Computer scientists will be trying to find one, or to improve the bound and show that no better algorithm is possible. We'll consider some such problems next week.

Divide-and-conquer sorting

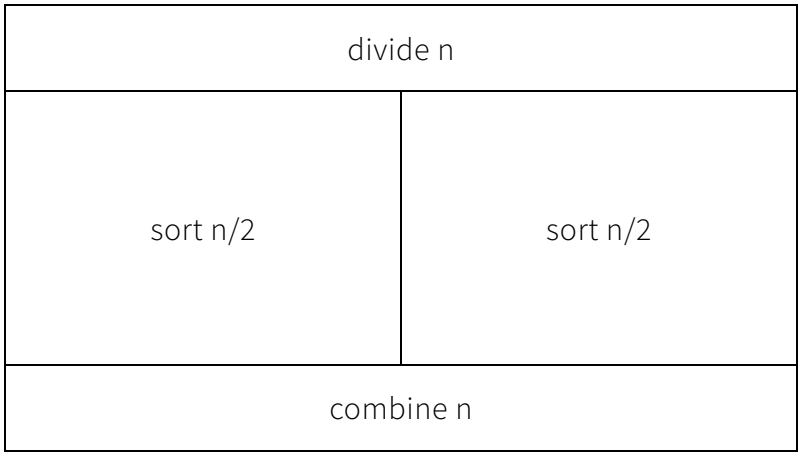
Recall the *divide-and-conquer* strategy:

- If the input is small, compute the answer using a direct method.
- Otherwise:
  - Divide the problem into smaller subproblems
  - Solve each subproblem recursively
  - Combine the solutions

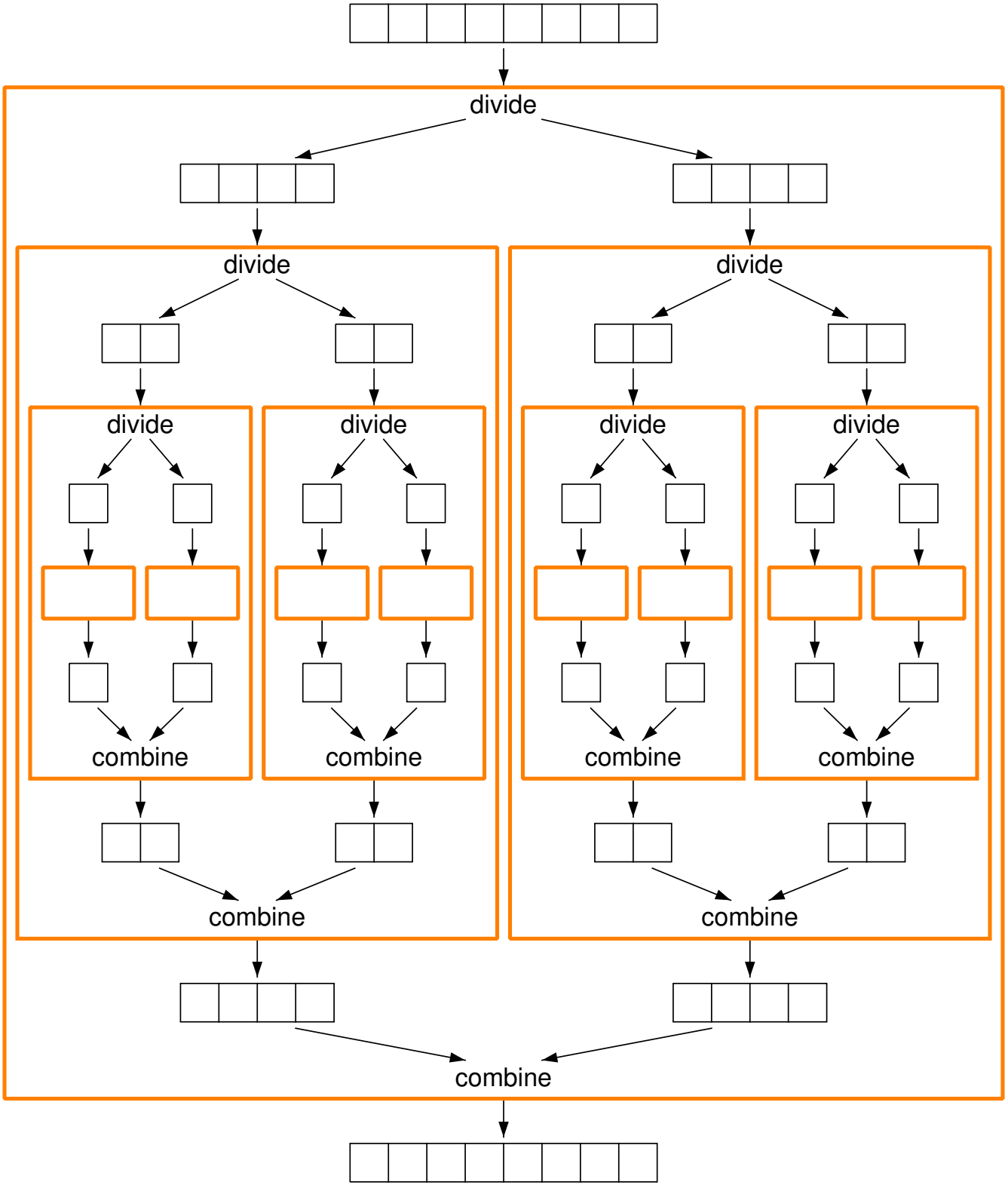
Let's apply this strategy to sorting:

- An array of length 0 or 1 is already ordered, so there is nothing to do.
- Given a longer array, we want to divide the elements between two subarrays, sort those (recursively), and

combine the results into a sorted array.



We'll be using divide and combine methods that take at most O(n) time. How much time will the resulting algorithm take? If the division is evenly balanced at each stage, the work done can be diagrammed like this:



Each row takes O(n) time, and there are O(log n) rows, so the total time is O(n log n). As we have seen above, this is asymptotically optimal for sorting. But this happy outcome relies on a balanced split of the array.

We shall consider two schemes for dividing and combining:

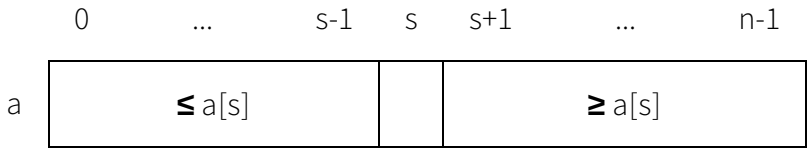
- In quicksort, the divide phase does most of the work, and the combine phase is trivial.
- In mergesort, the divide phase is trivial, and most of the work is done in the combine phase.

The quicksort algorithm was invented in 1960 by Tony Hoare, who described the algorithm elegantly in the Algol 60 *programming language using recursion.*

> « there are two ways of constructing a software design: One way is to make it so simple that there are *obviously* no deficiencies and the other way is to make it so complicated that there are no *obvious* deficiencies.

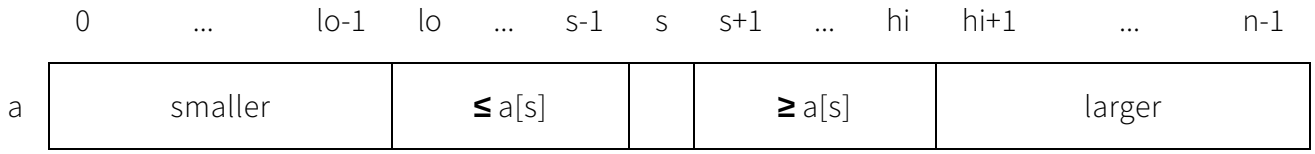
Quicksort

To use the divide-and-conquer strategy, we need a way of dividing the array in two. One way is to pick an element and re-arrange the array so that all the elements smaller than the chosen element are moved to its left, and all those larger are moved to its right. This is called *partitioning* the array, and the chosen element is called the *pivot*. This leaves the pivot in its final position, which depends on how many elements are smaller or larger:



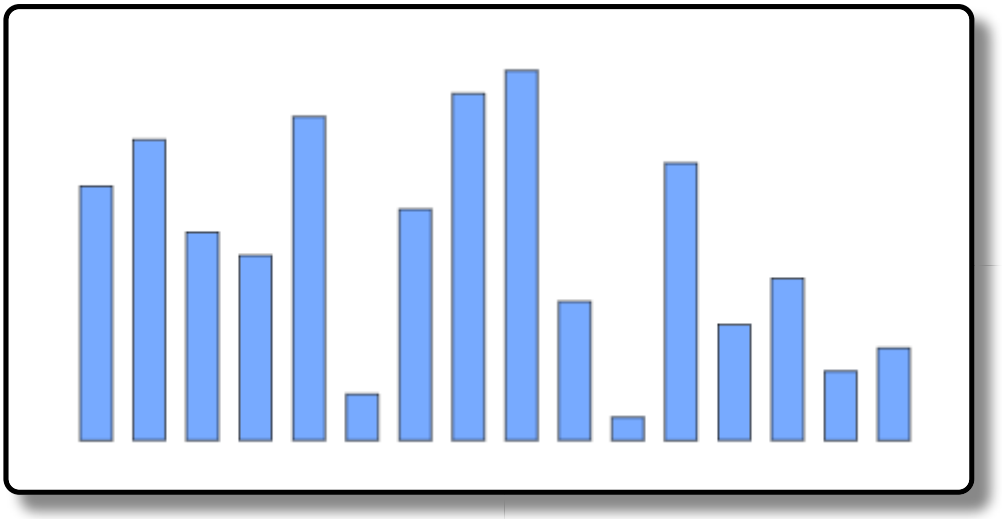
Once we have split the array in this way, we can sort the two subarrays (using the same method), and then the whole array will be sorted.

In general, we will be partitioning a part of the array, marked out by variables `lo` and `hi`:



To get a feel for the process, try to apply the above strategy to sort the array below using the applet below. The elements that have reached their final positions are in grey.

- Click on an element in one of the unsorted sections to select it as a pivot. The section of unsorted elements around the pivot will be re-arranged (by swapping) so that the smaller ones are left of the pivot and the larger ones to the right.
- Pressing Reset re-shuffles the array.



Try picking a middling value from each section. What happens if you pick the largest or smallest value in the section?

Partitioning an array

The first step is a partitioning method that selects a pivot from between positions `lo` and `hi` (inclusive) in the array `a`, partitions that section of the array using the pivot, and returns the final position of the pivot. A simple strategy for choosing the pivot is to take the leftmost element.

The partition algorithm separates the elements less than and greater than the pivot as follows:

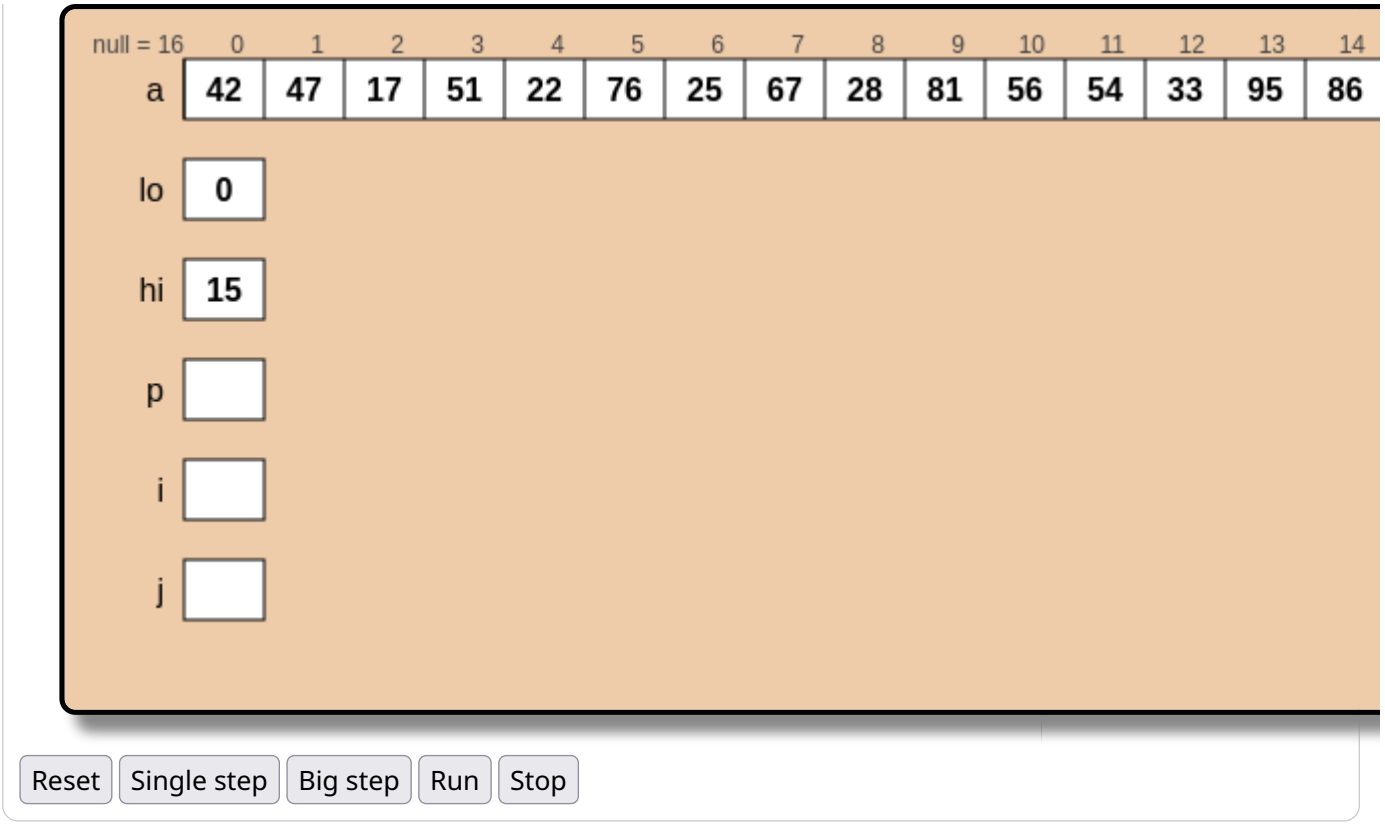
- Until the elements are separated,
  - Move in from the left until an element larger than the pivot is found.
  - Move in from the right until an element smaller than the pivot is found.
  - Swap these two elements.
- Finally, swap the pivot into position between the two groups.

```
Algorithm Partition(a, lo, hi)

    p ← a[lo]
    i ← lo + 1
    j ← hi

        invariant: lo+1 ≤ i ≤ j ≤ hi ≤ n AND a[lo+1..i-1] ≤
p ≤ a[j+1..hi]
    WHILE i ≤ j

            invariant: lo+1 ≤ i ≤ j+1 AND a[lo+1..i-1] ≤
p

        WHILE i ≤ j AND a[i] ≤ p
            i ← i + 1

            invariant: i-1 ≤ j ≤ hi AND p ≤ a[lo+1..hi] ≤
p

        WHILE j ≥ i AND a[j] ≥ p
            j ← j - 1
        IF i < j
            Swap(a, i, j)
            i ← i + 1
            j ← j - 1
    Swap(a, lo, j)
    RETURN j
```

This algorithm has two important properties:

- It updates the section of the array from `lo` to `hi` in place, using O(1) extra space.
- It takes time proportional to the size of the subarray (i.e. hi-lo+1).

Try the partition algorithm:



Example

Let's consider how we might sort this array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
42	47	17	51	22	76	25	67	28	81	56	54	33	95	86	36

Choose the first value in the array as the pivot and partition the array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
25	36	17	33	22	28	42	67	76	81	56	54	51	95	86	47

Now all that remains is to sort the subarray to the left and the one to the right. Once we've done that, the whole array will be sorted. So we sort each subarray using the same procedure. For the left subarray we use 25 as the pivot. For the right one, we use 67:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
17	22	25	33	36	28	42	54	47	51	56	67	81	95	86	76

Now each of those subarrays has been divided into two, and we continue in the same way, until the subarrays are small:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
17	22	25	28	33	36	42	51	47	54	56	67	76	81	86	95

Here most of the subarrays are of length 1, and thus trivially sorted. For other subarrays, we must partition again:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
17	22	25	28	33	36	42	47	51	54	56	67	76	81	86	95

Now the remaining subarrays are of length 1, and thus trivially sorted, and the whole array is sorted:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
17	22	25	28	33	36	42	47	51	54	56	67	76	81	86	95

The recursive quicksort algorithm

Now the algorithm to sort a section of an array from `lo` to `hi` is a divide-and-conquer scheme, using Partition as the divide phase:

Algorithm QSort(a, lo, hi)

```
IF lo < hi
    s ← Partition(a, lo, hi)
    QSort(a, lo, s - 1)
    QSort(a, s + 1, hi)
```

There is nothing to do in the combine phase, because the sorted subarrays are already in the correct positions, to the left and right of the pivot.

To sort the whole array, we call the recursive procedure on the whole range:

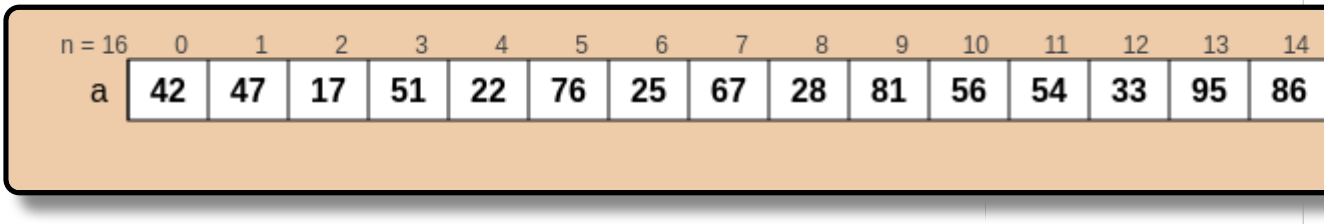
Algorithm QuickSort(a[0..n-1])

```
QSort(a, 0, n - 1)
```

A trace of the above example:

- QSort(a, 0, 15)
  - Partition(a, 0, 15) returns 6
  - QSort(a, 0, 5)
    - Partition(a, 0, 5) returns 2
    - QSort(a, 0, 1)
      - Partition(a, 0, 1) returns 0
      - QSort(a, 0, -1)
      - QSort(a, 1, 1)
    - QSort(a, 3, 5)
      - Partition(a, 3, 5) returns 4
      - QSort(a, 3, 3)
      - QSort(a, 5, 5)
  - QSort(a, 7, 15)
    - Partition(a, 7, 15) returns 11
    - QSort(a, 7, 10)
      - ...
    - QSort(a, 12, 15)
      - ...

Try the quicksort algorithm.
Note: Each of these activations is operating on the *same* array. This is marked in the applet by a connecting line on the right.



Reset | Single step | Big step | Run | Stop

Analysis

The amount of work done is proportional to the amount of partitioning (the number of white squares in the above walkthrough). If we have a reasonably balanced split (and on average we do), we have O(log n) levels of recursion, with O(n) partitioning work on each level, for a total of O(n log n).

Worst case

The chief defect of quicksort is its worst case time performance. One way of triggering this is to use an array that is already ordered:

0	1	2	3	4	5
A	B	C	D	E	F

In such a case, choosing the first element of the subarray as a pivot will yield a very uneven split, with one subarray of length 0 and the other only 1 shorter than the original at each step:

A	B	C	D	E	F
A	B	C	D	E	F
A	B	C	D	E	F
A	B	C	D	E	F

A	B	C	D	E	F
A	B	C	D	E	F

With an ordered array of length n, this will have a depth of n, with a total cost of

$$n + (n-1) + (n-2) + \ldots + 2 + 1$$

From the calculation at the start of week 3, we know this is $O(n^2)$. So quicksort takes $O(n^2)$ time in the worst case.

This choice of the pivot is crucial to the performance of quicksort. Ideally we want a value that lies exactly in the middle (the median), so that the two subarrays have the same size. However finding the median takes too long.

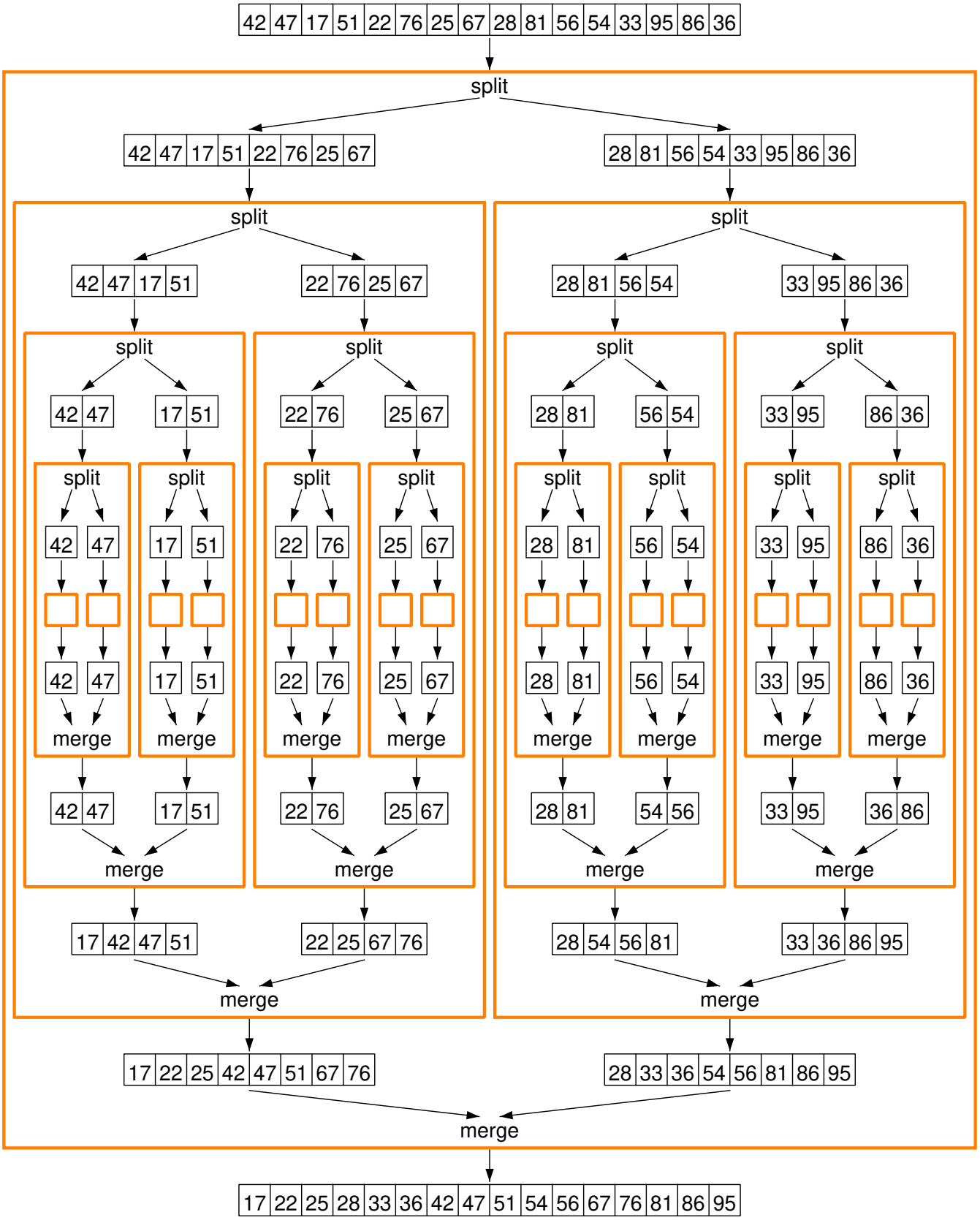
Achieving good performance through balanced splitting is a common theme of tree structures, which you'll meet in Data Structures and Algorithms next year.

Quicksort is often used in practice, because the average case is very fast, and the worst case can be made very rare by careful selection of the pivot. One common method is to take the median of the first, middle and last elements.

Mergesort

Let's try a different instantiation of the divide-and-conquer scheme, dividing the array in the simplest way possible: into the first half and the second half. Unlike with quicksort, the division does not depend on the data in the array, so we make it as even as possible. After approximately log n steps, we have subarrays of length 1, which are trivially sorted. We then repeatedly merge these to make longer sorted arrays until we have sorted the whole collection.

Applying this approach on the same array as before:



This is the opposite of quicksort in a sense, because here the divide part is trivial, and the combine (merge) does all the work.

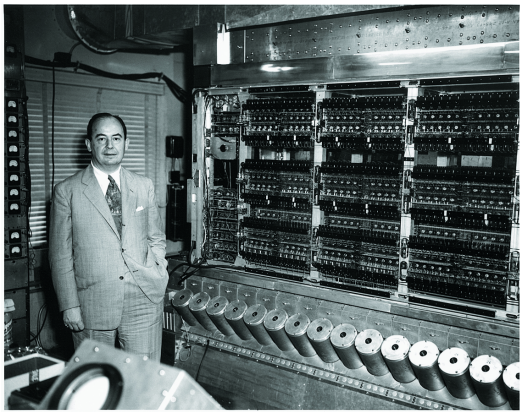
Merging two sorted arrays

The central operation here is merging:

- Since the two collections to be merged are already in order, we need only pass through them once each. Thus the time is proportional to the lengths of the lists being merged.
- In general, it is not possible to merge two arrays in place. A new one must be created.

Algorithm Merge(l[0..nl-1], r[0..nr-1])

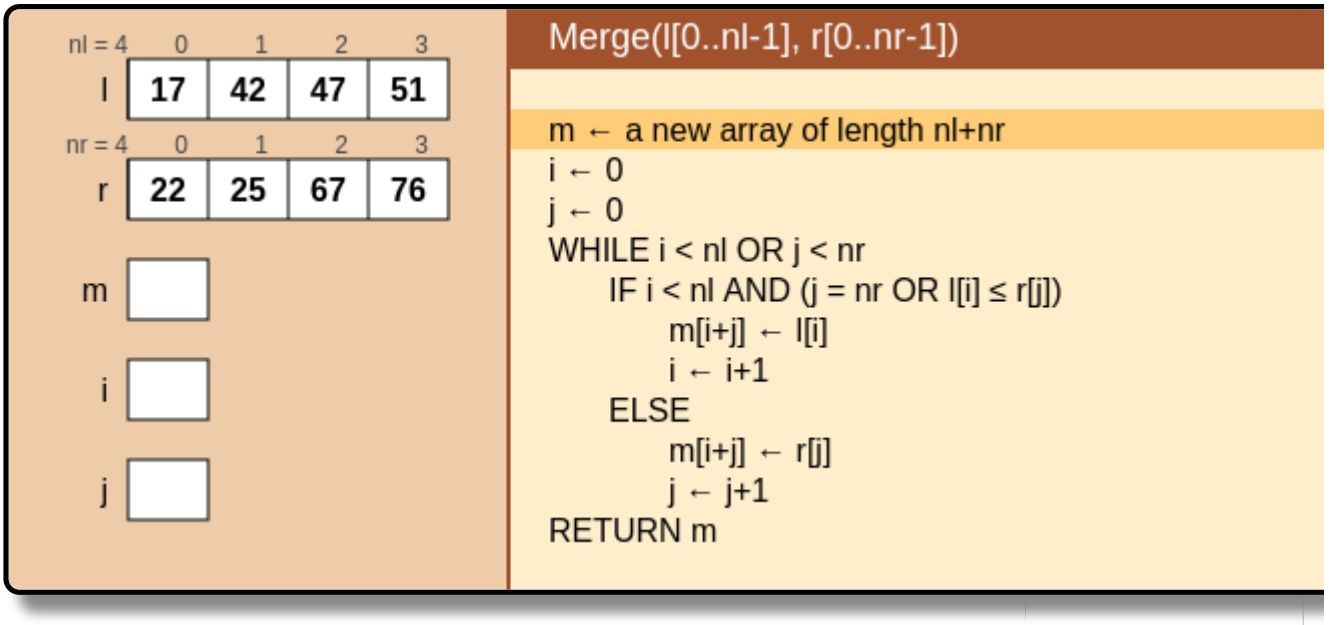
```
m ← a new array of length nl+nr
i ← 0
j ← 0
    invariant: i ≤ nl AND j ≤ nr AND
    m contains the first i+j elements of the merge of l and r
WHILE i < nl OR j < nr
    IF i < nl AND (j = nr OR l[i] ≤ r[j])
        m[i+j] ← l[i]
        i ← i+1
    ELSE
        m[i+j] ← r[j]
        j ← j+1
RETURN m
```



The merge sort algorithm was invented in 1945 by John von Neumann, a mathematician who made many contributions to computer architecture. (Here seen at Princeton in 1952 with part of the latest 40kbit computer)

> It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in five years. -- John von Neumann, 1949

Try the merge algorithm:



Reset | Single step | Big step | Run | Stop

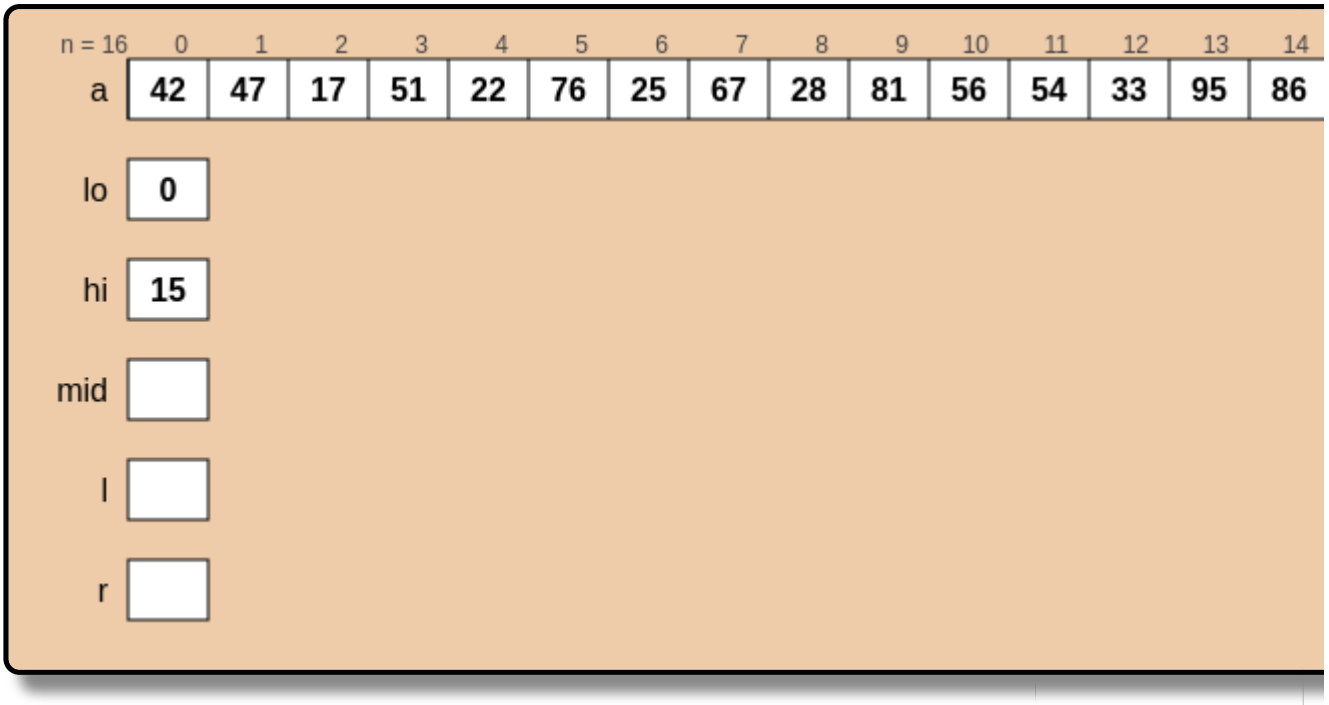
The recursive mergesort algorithm

Now the algorithm to an array is another divide-and-conquer scheme, this time using Merge as the combine phase. The divide phase is just calculating the middle index of the array.

Algorithm MergeSort(a[0..n-1], lo, hi)

```
IF lo ≥ hi
        RETURN a[lo..hi]
mid ← (lo + hi + 1) DIV 2
l ← MergeSort(a, lo, mid-1)
r ← MergeSort(a, mid, hi)
RETURN Merge(l, r)
```

Try the merge sort algorithm:



Reset | Single step | Big step | Run | Stop

Here's a trace on the above array:

MergeSort([42, 47, 17, 51, 22, 76, 25, 67, 28, 81, 56, 54, 33, 95, 86, 36])

- MergeSort([42, 47, 17, 51, 22, 76, 25, 67])
  - MergeSort([42, 47, 17, 51])
    - MergeSort([42, 47])
      - MergeSort([42])
        - returns [42]
      - MergeSort([47])
        - returns [47]
      - returns [42, 47]
    - MergeSort([17, 51])
      - MergeSort([17])
        - returns [17]
      - MergeSort([51])
        - returns [51]
      - returns [17, 51]
    - returns [17, 42, 47, 51]
  - MergeSort([22, 76, 25, 67])
    - ...
    - returns [22, 25, 67, 76]
  - returns [17, 22, 25, 42, 47, 51, 67, 76]

Note that each call to MergeSort ends with a call to Merge, the result of which is returned. The call Merge([17,42,47,51], [22,25,67,76]) is tabulated below:

i	j	m
0	0	
1	0	17
1	1	17 22
1	2	17 22 25
2	2	17 22 25 42
3	2	17 22 25 42 47
4	2	17 22 25 42 47 51
4	3	17 22 25 42 47 51 67

> MergeSort([28, 81, 56, 54, 33, 95, 86, 36])
>
> > ...
> >
> > returns [28, 33, 36, 54, 56, 81, 86, 95]
>
> returns [17, 22, 25, 28, 33, 36, 42, 47, 51, 54, 56, 67, 76, 81, 86, 95]

4 4

17	22	25	42	47	51	67	76

Analysis

In the worst case, the depth of the recursion in approximately log n, with O(n) work on each level, for a total of O(n log n) time, which matches the theoretical lower bound.

However, because merging cannot be done in place, merge sort requires O(n) extra space.

Comparison of sorting algorithms

Let's sum up by comparing the sorting algorithms we've seen.

Recall from week 3:

definition

An adaptive sort is one that performs better on partially sorted inputs, and takes O(n) time on sorted or almost-sorted inputs.

definition

A stable sort is one that preserves the relative ordering of elements that compare as equal.

The table below also includes heap sort, a method using a data structure called a *heap* (not the area of memory of that name), which will be covered in Data Structures and Algorithms next year.

	extra space	average time	worst time	adaptive?	stable?
selection sort	O(1)	$O(n^2)$	$O(n^2)$	no	no
insertion sort	O(1)	$O(n^2)$	$O(n^2)$	yes	yes
shell sort	O(1)	near optimal	near optimal	no	no
quicksort	O(log n)	O(n log n)	$O(n^2)$	no	not usually
merge sort	O(n)	O(n log n)	O(n log n)	can be	yes

heap sort	O(1)	O(n log n)	O(n log n)	no	no

In practice quicksort is often used, despite its terrible worst case, because it has better constant factors that the others, and with care the worst case can be made very rare.
